

TIVIDY

The team

Tamlyn Khan (u)
Yoshua Smit (u25433726)
Isabelle Engelbrecht (u25090501)
Jonathan Witter (u25108582)
Sudud Abughazaleh (u25282728)

Github Repo Link:

<https://github.com/IridescentDucky/COS214-Practical-6-Project>

Workflow-management research

- **What is a workflow and what is a workflow management system?**

A workflow is an automated process where tasks pass between people according to set rules. A WfMS stores, runs and manages these processes (Hollingsworth, 1995; WfMC, 1996). So TIVIDY's engine must run processes without containing them.

- **What is the difference between a workflow definition and a running workflow instance?**

A definition is the template of steps, order and roles; an instance is one run of it, with its own data and history (WfMC, 1996). So these need separate classes, and changing one instance must not affect the others.

- **How are activities or work items represented?**

Activities create work items that go to a worklist and move through states such as waiting, active and completed (WfMC, 1996). So work items need a state-based lifecycle.

- **How are work items assigned and routed?**

Routing uses sequence, parallel, choice and loops, and each product supports a different subset (van der Aalst et al., 2003). Work is either pushed to someone by role or workload, or pulled from a shared list (Russell et al., 2005). So both routing and assignment must be swappable, not hard-coded.

- **How are approvals, rejection and escalation handled?**

An approval is a work item whose result decides the next step. Rejection sends work back, and escalation reassigns it when a deadline passes (WfMC, 1996; Russell et al., 2005). So deadlines and escalation rules must be configurable.

- **How are participants, roles and responsibilities represented?**

Definitions name roles, not people, and the engine picks a person at run time from the organisation's structure (WfMC, 1996). So staff changes should not require changing definitions.

- **How are dependencies, events and failures handled?**

Steps wait on conditions or joins, events trigger actions, and each type of failure gets a chosen response such as retry, skip or undo (WfMC, 1996; Russell, van der Aalst and ter Hofstede, 2006). So many parts must be able to react to one event, and failure handling must be configurable.

- **How do workflow systems interact with external systems?**

Engines reach outside systems through a separate interface (Hollingsworth, 1995); Camunda, for example, queues jobs for external workers to pick up (Camunda, n.d.b). So TIVIDY needs a translation layer for each external system.

- **How is workflow history or audit information recorded?**

Every state change is logged with a timestamp (WfMC, 1996). So actions should be stored as history objects, which also makes undo possible.

- **How are running workflows monitored and reported on?**

Monitoring is separate from execution, and logs are analysed afterwards to find bottlenecks (Hollingsworth, 1995; van der Aalst, 2016). So reporting should read the history from outside the core classes.

- **What makes a workflow engine reusable across different processes?**

All process rules live in the definition, not the engine (WfMC, 1996). So no scenario-specific code belongs in the engine.

- **Which information belongs to a workflow definition and which belongs to a running instance?**

Definition: steps, order, conditions, roles and deadlines. Instance: current state, data, assigned people and history (WfMC, 1996).

- **How do real workflow systems model nested activities or sub-processes?**

A sub-process has its own definition and can be nested inside others (WfMC, 1996). So nested work forms a tree the engine can traverse.

- **Which decisions are normally configurable rather than hard-coded into the workflow engine?**

Routing, assignment, escalation and failure handling (WfMC, 1996; Russell et al., 2005; Russell, van der Aalst and ter Hofstede, 2006).

- **How are changes to workflow definitions handled while work is already in progress?**

A change creates a new version, and running instances stay on the old one unless they can be safely migrated (Camunda, n.d.a; Rinderle, Reichert and Dadam, 2004). So definitions must be versioned.

References

Camunda (n.d.a) Process versioning. Available at:
<https://docs.camunda.org/manual/latest/user-guide/process-engine/process-versioning>
(Accessed: 1 October 2026).

Camunda (n.d.b) *External tasks*. Available at:
<https://docs.camunda.org/manual/latest/user-guide/process-engine/external-tasks>
(Accessed: 1 October 2026).

Hollingsworth, D. (1995) *The workflow reference model*. Brussels: Workflow Management Coalition.

Rinderle, S., Reichert, M. and Dadam, P. (2004) 'Correctness criteria for dynamic changes in workflow systems: a survey', *Data & Knowledge Engineering*, 50(1), pp. 9–34.

Russell, N., van der Aalst, W.M.P. and ter Hofstede, A.H.M. (2006) 'Workflow exception patterns', in *CAiSE 2006*. Berlin: Springer, pp. 288–302.

Russell, N., van der Aalst, W.M.P., ter Hofstede, A.H.M. and Edmond, D. (2005) 'Workflow resource patterns', in *CAiSE 2005*. Berlin: Springer, pp. 216–232.

van der Aalst, W.M.P. (2016) *Process mining: data science in action*. 2nd edn. Berlin: Springer.

van der Aalst, W.M.P., ter Hofstede, A.H.M., Kiepuszewski, B. and Barros, A.P. (2003) 'Workflow patterns', *Distributed and Parallel Databases*, 14(1), pp. 5–51.

Workflow Management Coalition (WfMC) (1996) *Terminology & glossary*. Brussels: Workflow Management Coalition.

Scenario description

We have decided to implement an Exam Management System (EMS) using the TIVIDY (the workflow-management engine) that we have created. Our system governs the end-to-end lifecycle of exam papers, prioritising strict paper tracking and secure state management.

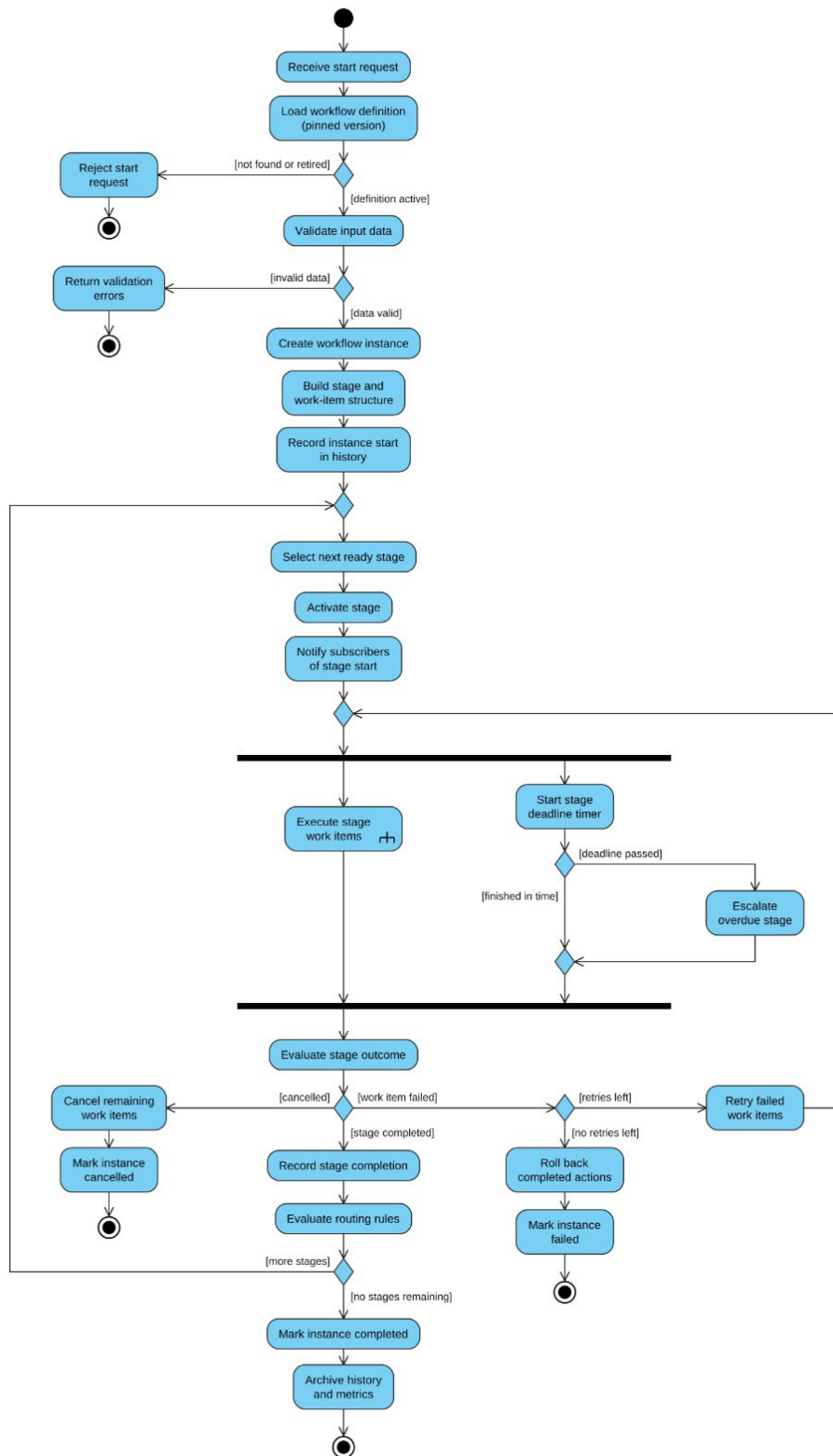
The system makes use of the Abstract Factory design pattern to create the papers that the students will write. We make use of the prototype pattern to make clones of the paper so that each student can have their own paper (this simulates printing papers). Once the paper is written by the student it has a variety of states that it goes through starting with *WrittenState*. The Observer pattern is used to determine when the paper has been delivered to the markers and will notify the paper to change its state. From there markers will be able to set the paper into a *MarkingState*, *PausedState*, *MarkedState* and *CapturedState*. The markers also make use of the Chain of Responsibility pattern to have a hierarchy between tutors, ALs, lecturers and module coordinators.

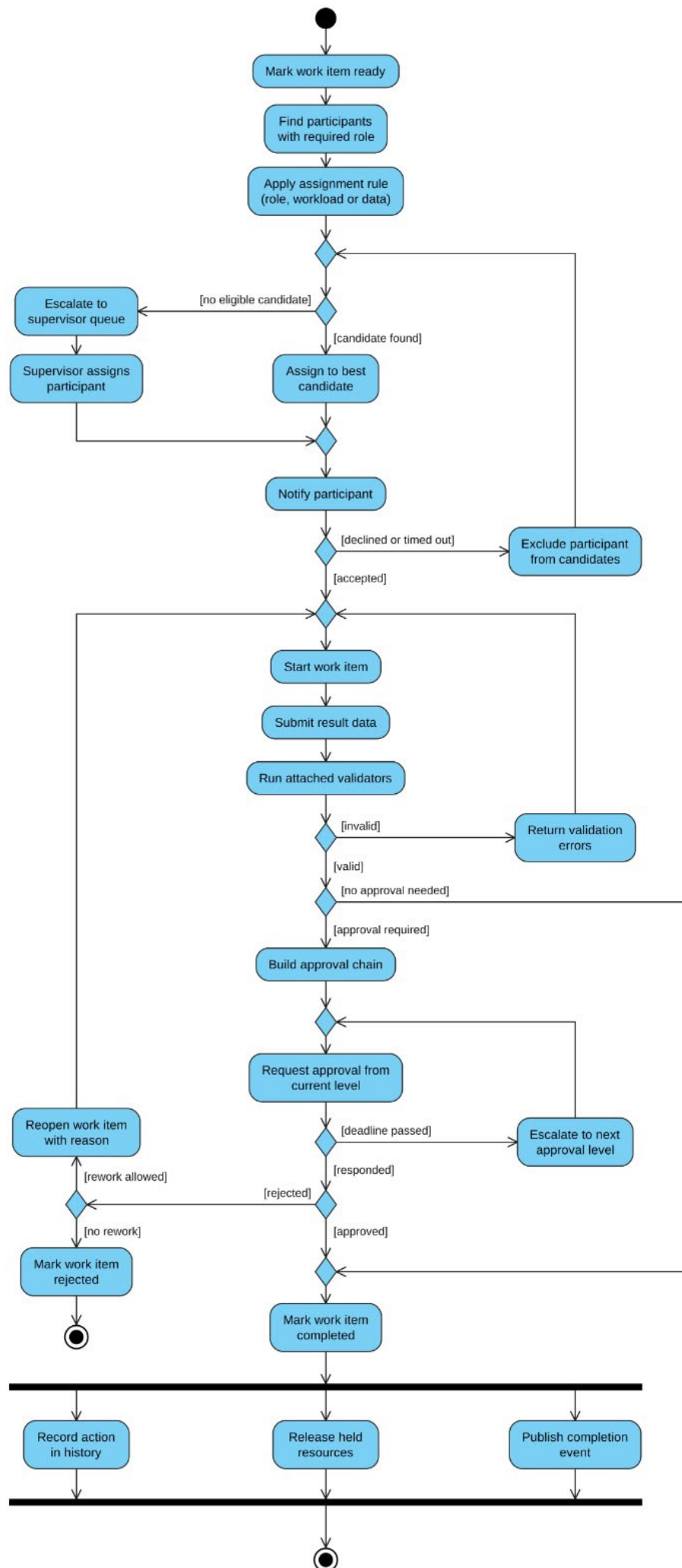
GoF participant mapping

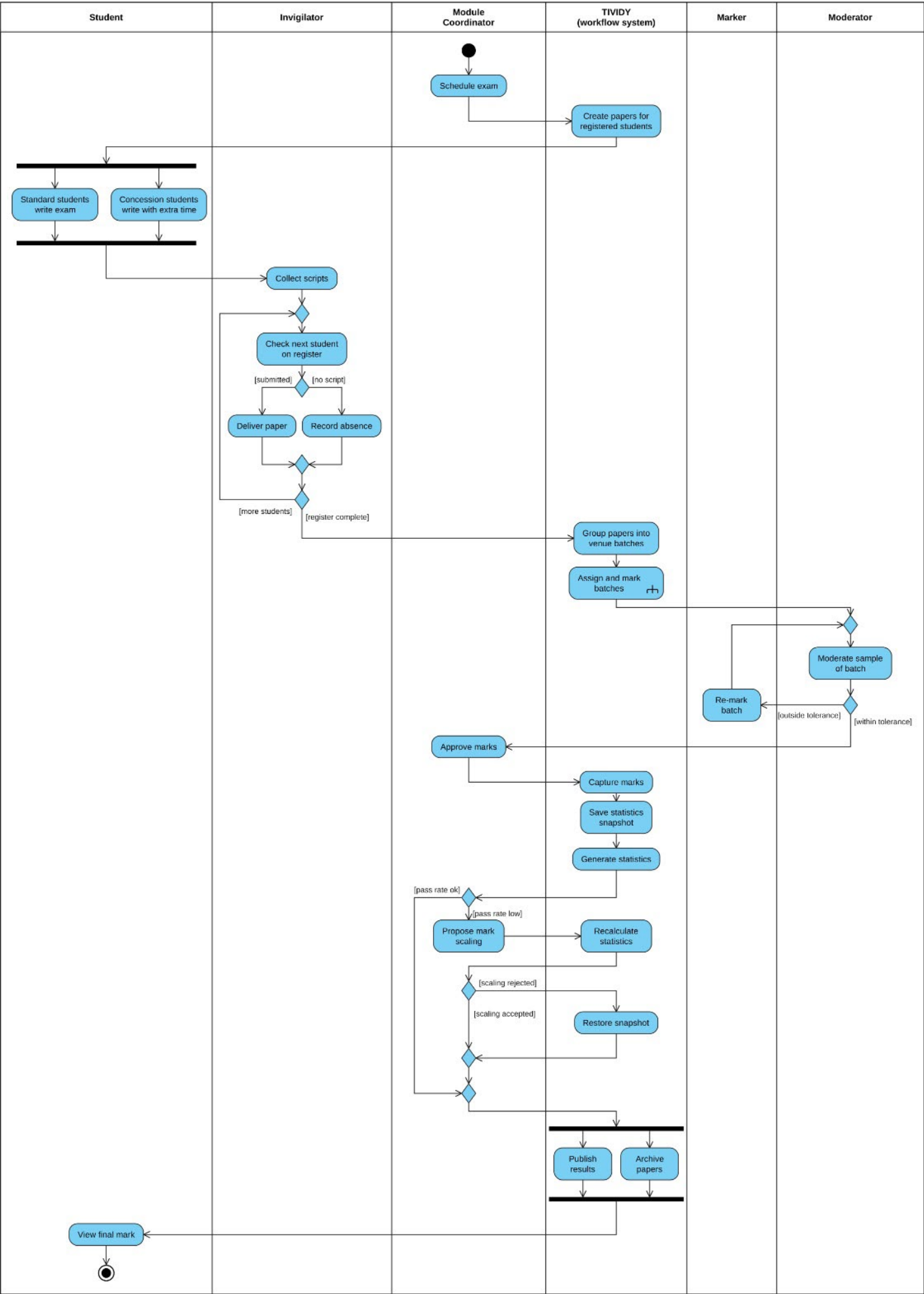
Pattern Justifications

Initial TIVIDY system scope and design rationale

UML Activity Diagrams



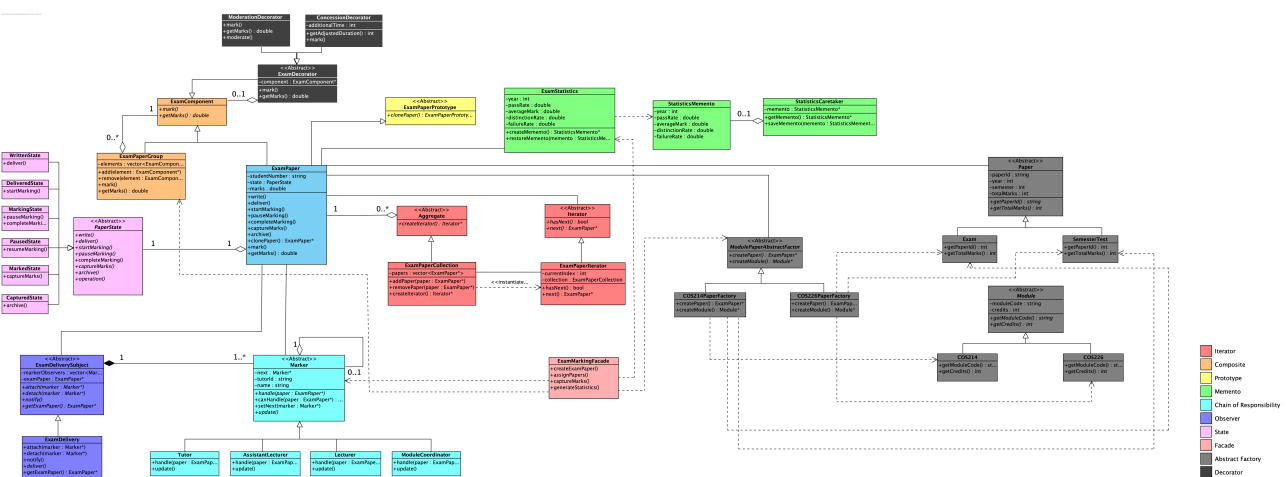




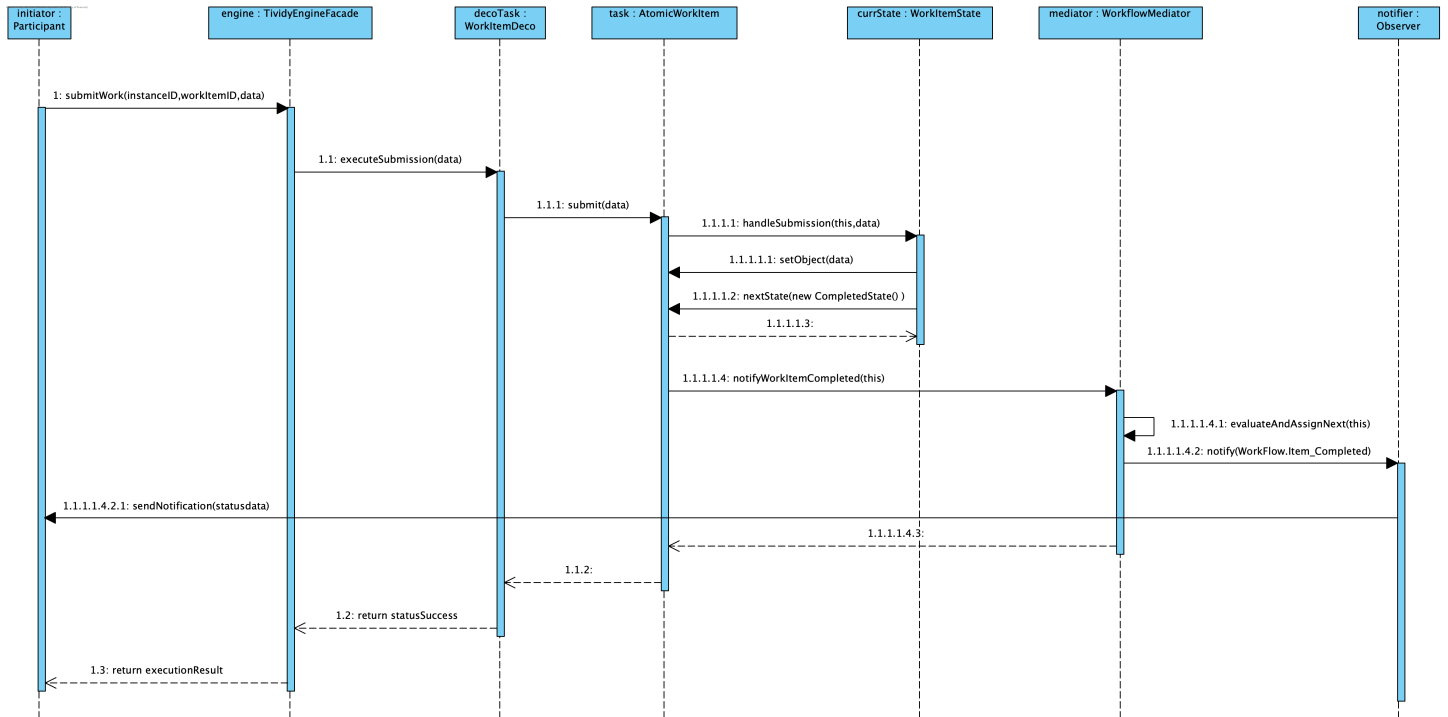
UML Class Diagrams

TIVIDY UML class diagram

Scenario UML class diagram

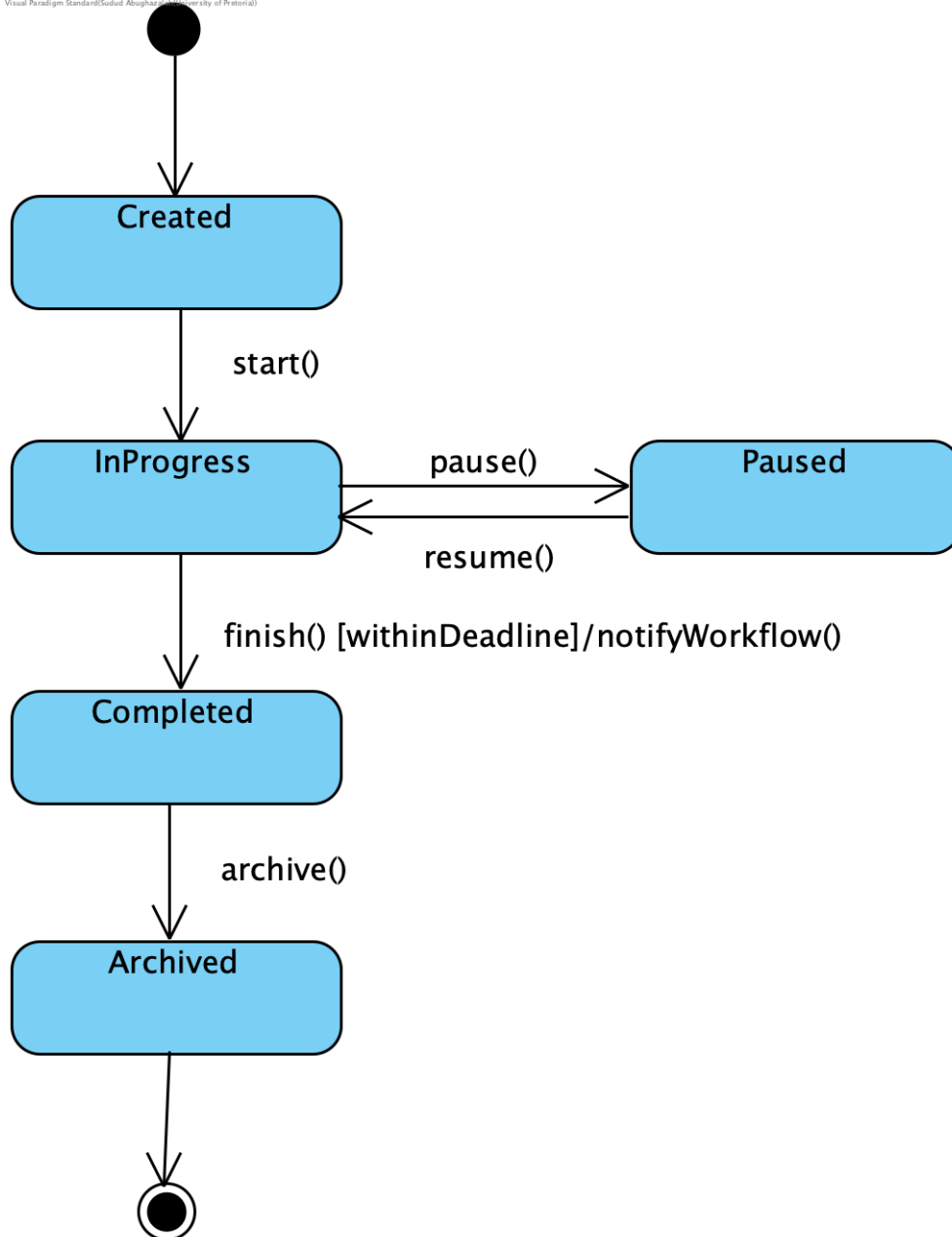


UML Sequence Diagrams



UML State Diagrams

Visual Paradigm Standard(Sudod Aboghuza, University of Pretoria)



Design decisions and revision history

Team contribution statement